Lab 7(B) - Mathematical and Statistical Filters

In [ ]:
```
import cv2
import numpy as np

from IPython.display import Image, display
from matplotlib import pyplot as plt

plt.rc('text', usetex=False)
```

In [ ]:
```
imgPath = 'noise.jpg'
img = cv2.imread(imgPath, 0)

def add_noise(image, noise_type):
    row, col = image.shape

    if noise_type == "gaussian":
        mean = 0
        var = 0.01
        sigma = var**0.5
        gauss = np.random.normal(mean, sigma, (row, col))
        noisy = image / 255 + gauss
        return np.clip(noisy * 255, 0, 255).astype(np.uint8)

    elif noise_type == "salt_pepper":
        s_vs_p = 0.5
        amount = 0.04
        noisy = np.copy(image)
        # Salt (white pixels)
        num_salt = np.ceil(amount * image.size * s_vs_p)
        coords = [np.random.randint(0, i - 1, int(num_salt)) for i in
image.shape]
        noisy[tuple(coords)] = 255
        # Pepper (black pixels)
        num_pepper = np.ceil(amount * image.size * (1.0 - s_vs_p))
        coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in
image.shape]
        noisy[tuple(coords)] = 0
        return noisy

    elif noise_type == "exponential":

        expo = np.random.exponential(scale=10.0, size=(row, col))
        noisy = image + expo
        return np.clip(noisy, 0, 255).astype(np.uint8)

    elif noise_type == "uniform":
        low = -20
        high = 20
        uni = np.random.uniform(low, high, (row, col))
```

```
        noisy = image + uni
        return np.clip(noisy, 0, 255).astype(np.uint8)



img = cv2.imread(imgPath, 0)

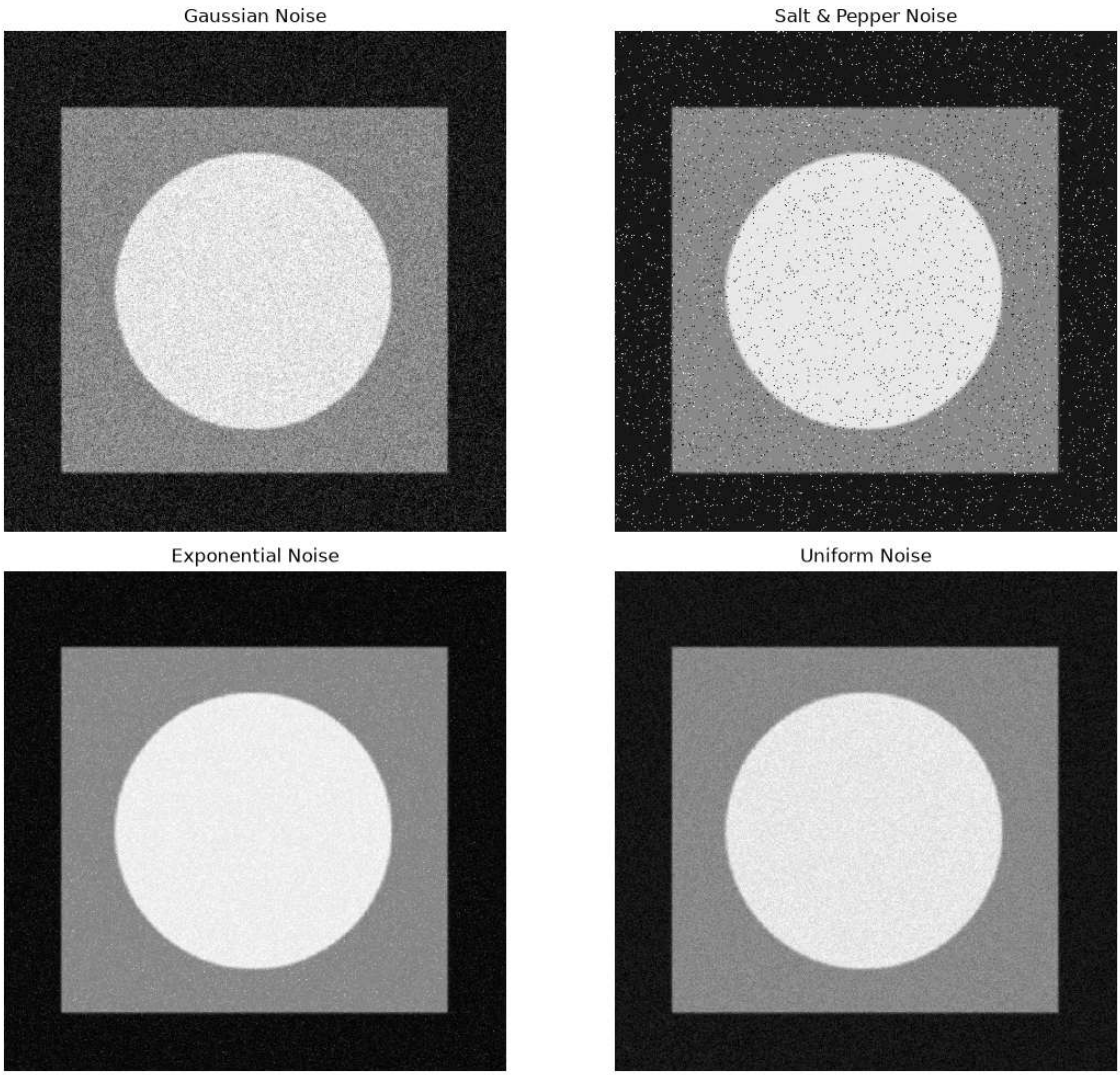
noise_gauss = add_noise(img, "gaussian")
noise_sp = add_noise(img, "salt_pepper")
noise_expo = add_noise(img, "exponential")
noise_uni = add_noise(img, "uniform")

plt.figure(figsize=(12, 10))

plt.subplot(221), plt.imshow(noise_gauss, cmap='gray'), plt.title('Gaussian
Noise')
plt.subplot(222), plt.imshow(noise_sp, cmap='gray'), plt.title('Salt &
Pepper Noise')
plt.subplot(223), plt.imshow(noise_expo, cmap='gray'),
plt.title('Exponential Noise')
plt.subplot(224), plt.imshow(noise_uni, cmap='gray'), plt.title('Uniform
Noise')

for i in range(1, 5):
    plt.subplot(220 + i).axis('off')

plt.tight_layout()
plt.show()
```



In [ ]:

```
filtered_img = cv2.blur(noise_gauss, (3, 3))

display(filtered_img)
```

```
array([[ 7, 17, 18, ...,  7, 10, 10],
       [25, 26, 19, ..., 14, 18, 16],
       [29, 30, 16, ..., 15, 22, 23],
       ...,
       [28, 24, 25, ..., 20, 19, 18],
       [29, 23, 25, ..., 18, 18, 13],
       [31, 20, 29, ..., 18, 16, 12]], shape=(506, 508), dtype=uint8)
```

In [ ]:

```
kernel_size = 3
kernel = np.ones((kernel_size, kernel_size), np.float32) / (kernel_size *
kernel_size)
arithmetic_mean_manual = cv2.filter2D(img, -1, kernel)

display(arithmetic_mean_manual)
```

```
array([[24, 24, 24, ..., 24, 24, 24],
       [24, 24, 24, ..., 24, 24, 24],
       [24, 24, 24, ..., 24, 24, 24],
       ...,
       [24, 24, 24, ..., 24, 24, 24],
       [24, 24, 24, ..., 24, 24, 24],
       [24, 24, 24, ..., 24, 24, 24]], shape=(506, 508), dtype=uint8)
```

Geometric Means

In [ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def geometric_mean_filter(image, kernel_size=3):
    img_float = image.astype(np.float64) + 1e-6

    log_img = np.log(img_float)

    kernel = np.ones((kernel_size, kernel_size), np.float32) / (kernel_size
* kernel_size)
    mean_log = cv2.filter2D(log_img, -1, kernel)

    geo_mean = np.exp(mean_log)

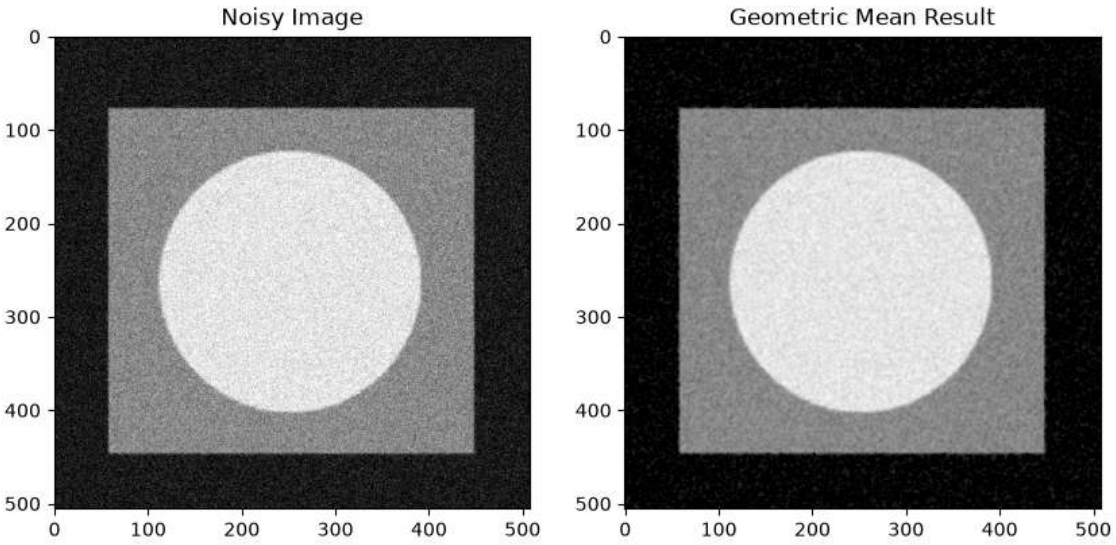
    return np.clip(geo_mean, 0, 255).astype(np.uint8)




noise_img = noise_gauss

result = geometric_mean_filter(noise_img, kernel_size=3)

plt.figure(figsize=(10, 5))
plt.subplot(121), plt.imshow(noise_img, cmap='gray'), plt.title('Noisy
Image')
plt.subplot(122), plt.imshow(result, cmap='gray'), plt.title('Geometric Mean
Result')
plt.show()
```



Harmonic Means Filter

In [ ]:

```
def harmonic_mean_filter(image, kernel_size=3):

    img_float = image.astype(np.float64) + 1e-6

    reciprocal_img = 1.0 / img_float


    kernel = np.ones((kernel_size, kernel_size), np.float32)
    sum_reciprocal = cv2.filter2D(reciprocal_img, -1, kernel)


    n = kernel_size * kernel_size
    harmonic_mean = n / sum_reciprocal

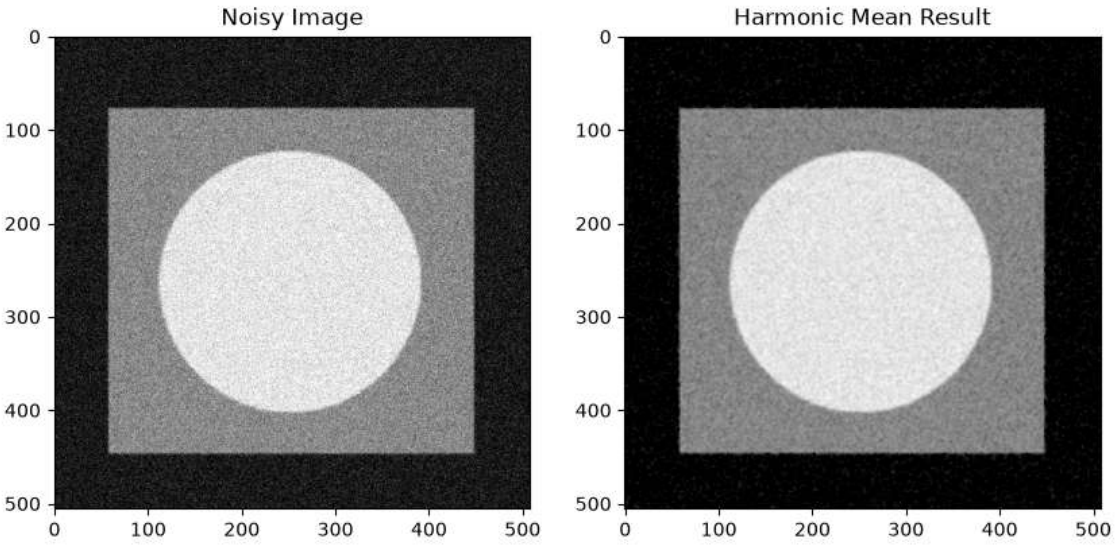
    return np.clip(harmonic_mean, 0, 255).astype(np.uint8)




noise_img = noise_gauss

cleaned_img = harmonic_mean_filter(noise_img, kernel_size=3)

plt.figure(figsize=(10, 5))
plt.subplot(121), plt.imshow(noise_img, cmap='gray'), plt.title('Noisy
Image')
plt.subplot(122), plt.imshow(cleaned_img, cmap='gray'), plt.title('Harmonic
Mean Result')
plt.show()
```



Contra - Harmonic Mean Filter

In [ ]:

```
def contra_harmonic_mean(image, kernel_size=3, Q=0):
    img_float = image.astype(np.float64) + 1e-6

    numerator = np.power(img_float, Q + 1)
    kernel = np.ones((kernel_size, kernel_size), np.float32)
    num_sum = cv2.filter2D(numerator, -1, kernel)

    denominator = np.power(img_float, Q)
    den_sum = cv2.filter2D(denominator, -1, kernel)


    result = num_sum / (den_sum + 1e-6)

    return np.clip(result, 0, 255).astype(np.uint8)

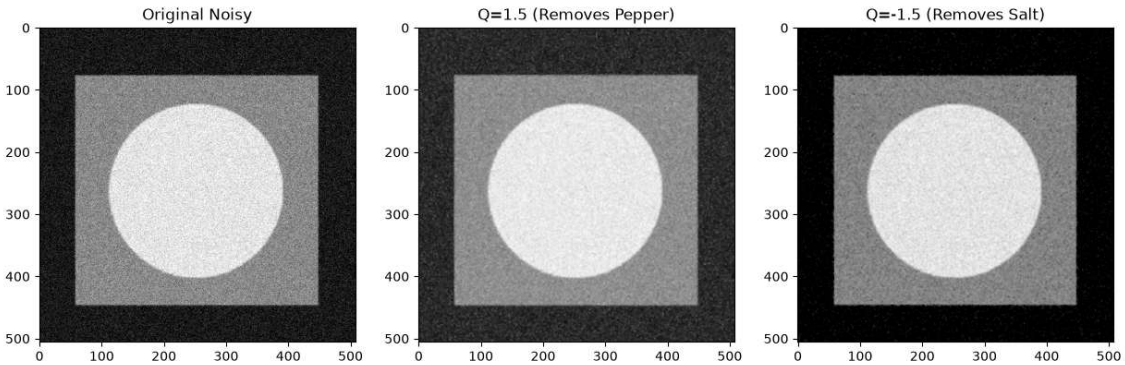


noise_img = noise_gauss


pepper_removed = contra_harmonic_mean(noise_img, kernel_size=3, Q=1.5)

salt_removed = contra_harmonic_mean(noise_img, kernel_size=3, Q=-1.5)

plt.figure(figsize=(15, 5))
plt.subplot(131), plt.imshow(noise_img, cmap='gray'), plt.title('Original
Noisy')
plt.subplot(132), plt.imshow(pepper_removed, cmap='gray'), plt.title('Q=1.5
(Removes Pepper)')
plt.subplot(133), plt.imshow(salt_removed, cmap='gray'), plt.title('Q=-1.5
(Removes Salt)')
plt.show()
```



Median Filter

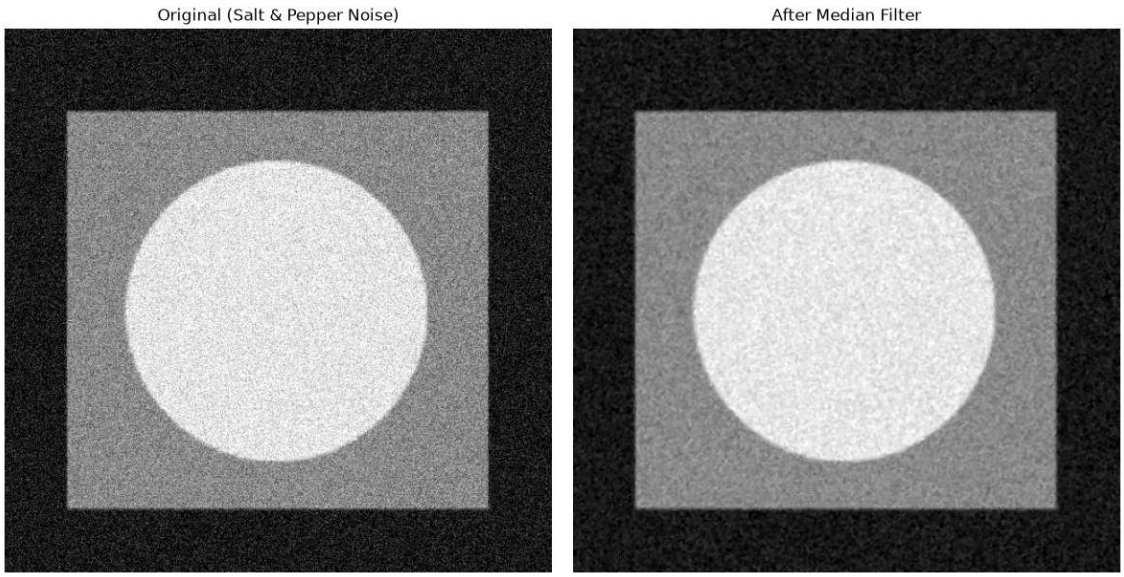
In [ ]:

```
noise_img = noise_gauss

median_result = cv2.medianBlur(noise_img, 3)

plt.figure(figsize=(12, 6))

plt.subplot(121), plt.imshow(noise_img, cmap='gray'), plt.title('Original
(Salt & Pepper Noise)'), plt.axis('off')
plt.subplot(122), plt.imshow(median_result, cmap='gray'), plt.title('After
Median Filter'), plt.axis('off')
plt.tight_layout()
plt.show()
```



Max and Min Filter

In [ ]:

```
noise_img = noise_gauss


kernel = np.ones((3, 3), np.uint8)

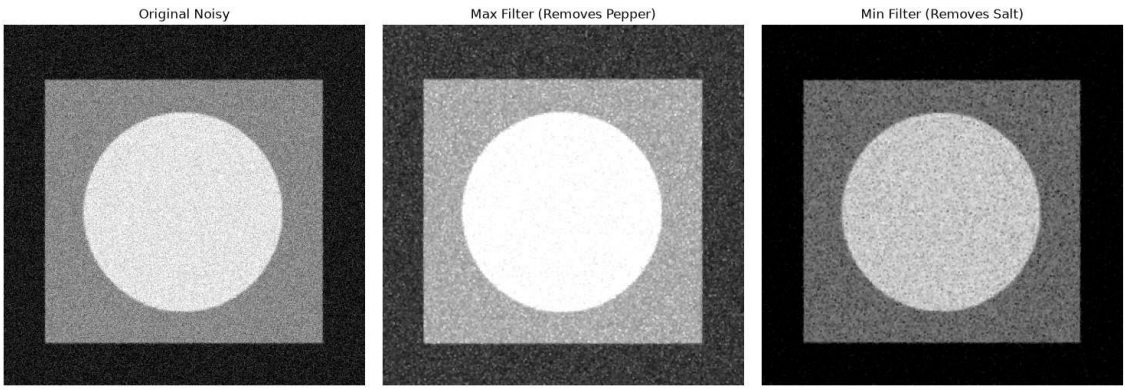

max_filter = cv2.dilate(noise_img, kernel)


min_filter = cv2.erode(noise_img, kernel)

plt.figure(figsize=(15, 5))

plt.subplot(131), plt.imshow(noise_img, cmap='gray'), plt.title('Original
Noisy')
plt.subplot(132), plt.imshow(max_filter, cmap='gray'), plt.title('Max Filter
(Removes Pepper)')
plt.subplot(133), plt.imshow(min_filter, cmap='gray'), plt.title('Min Filter
(Removes Salt)')

for ax in plt.gcf().axes: ax.axis('off')
plt.tight_layout()
plt.show()
```



Mid-Point Filtering

In [ ]:

```
def midpoint_filter(image, kernel_size=3):
    kernel = np.ones((kernel_size, kernel_size), np.uint8)

    max_img = cv2.dilate(image, kernel).astype(np.float64)

    min_img = cv2.erode(image, kernel).astype(np.float64)

    midpoint = (max_img + min_img) / 2

    return midpoint.astype(np.uint8)




noise_img = noise_gauss


result = midpoint_filter(noise_img, kernel_size=3)

plt.figure(figsize=(12, 6))

plt.subplot(121), plt.imshow(noise_img, cmap='gray'), plt.title('Original
(Gaussian/Uniform Noise)'), plt.axis('off')
plt.subplot(122), plt.imshow(result, cmap='gray'), plt.title('After Midpoint
Filter'), plt.axis('off')

plt.tight_layout()
plt.show()
```

